

Лабораторная работа № 6

Управление процессами

Павличенко Родион Андреевич

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Контрольные вопросы	13
4	Выводы	14
	Список литературы	15

Список иллюстраций

2.1	Создание процессов	6
2.2	Работа с процессами	7
2.3	Работа во втором терминале	7
2.4	Команда top	7
2.5	Создание процессов	8
2.6	Строки, где есть буквы dd	8
2.7	Уничтожение процесса	8
2.8	Задание 1	9
2.9	Задание 1	9
2.10	Задание 2	10
2.11	Задание 2	10
2.12	Задание 2	11
2.13	Задание 2	11
2.14	Задание 2	12
2.15	Задание 2	12

Список таблиц

1 Цель работы

Получить навыки управления процессами операционной системы.

2 Выполнение лабораторной работы

Получили полномочия администратора. Ввели следующие команды: `sleep 3600 &`; `dd if=/dev/zero of=/dev/null &`; `sleep 7200`. Ввели `Ctrl + z`, чтобы остановить процесс. Ввели `jobs`. Мы увидели три задания, которые мы только что запустили. Первые два имеют состояние `Running`, а последнее задание в настоящее время находится в состоянии `Stopped`. Для продолжения выполнения задания 3 в фоновом режиме ввели `bg 3`. С помощью команды `jobs` посмотрели изменения в статусе заданий.

```
[rapavlichenko@rapavlichenko ~]$ su -
Пароль:
[root@rapavlichenko ~]# sleep 3600 &
[1] 3666
[root@rapavlichenko ~]# dd if=/dev/zero of=/dev/null &
[2] 3677
[root@rapavlichenko ~]# sleep 7200
^Z
[3]+  Остановлен    sleep 7200
[root@rapavlichenko ~]# jobs
[1]  Запущен        sleep 3600 &
[2]-  Запущен        dd if=/dev/zero of=/dev/null &
[3]+  Остановлен    sleep 7200
[root@rapavlichenko ~]# bg 3
[3]+ sleep 7200 &
[root@rapavlichenko ~]# jobs
[1]  Запущен        sleep 3600 &
[2]-  Запущен        dd if=/dev/zero of=/dev/null &
[3]+  Запущен        sleep 7200 &
```

Рисунок 2.1: Создание процессов

Для перемещения задания 1 на передний план ввели `fg 1`. Ввели `Ctrl + c`, чтобы отменить задание 1. С помощью команды `jobs` посмотрели изменения в статусе заданий. Проделали то же самое для отмены заданий 2 и 3.

```

[root@rapavlichenko ~]# fg 1
sleep 3600
^C
[root@rapavlichenko ~]# jobs
[2]-  Запущен          dd if=/dev/zero of=/dev/null &
[3]+  Запущен          sleep 7200 &
[root@rapavlichenko ~]# fg 2
dd if=/dev/zero of=/dev/null
^C800157918+0 записей получено
800157917+0 записей отправлено
409680853504 байт (410 GB, 382 GiB) скопирован, 246,034 s, 1,7 GB/s
[root@rapavlichenko ~]# fg 3
sleep 7200
^C
[root@rapavlichenko ~]# jobs

```

Рисунок 2.2: Работа с процессами

Открыли второй терминал и под учётной записью своего пользователя ввели в нём: `dd if=/dev/zero of=/dev/null & 10`. Ввели `exit`, чтобы закрыть второй терминал.

```

[rapavlichenko@rapavlichenko ~]$ dd if=/dev/zero of=/dev/null &
[1] 3815
[rapavlichenko@rapavlichenko ~]$ exit

```

Рисунок 2.3: Работа во втором терминале

На другом терминале под учётной записью своего пользователя запустили `top`. Мы увидели, что задание `dd` всё ещё запущено. Для выхода из `top` использовали `q`. Вновь запустили `top` и в нём использовали `k`, чтобы убить задание `dd`. После этого вышли из `top`

```

[root@rapavlichenko ~]# top
top - 14:21:49 up 16 min,  2 users,  load average: 1,28, 1,10, 0,73
Tasks: 263 total,  2 running, 261 sleeping,  0 stopped,  0 zombie
%Cpu(s):  3,4 us,  5,6 sy,  0,0 ni, 90,9 id,  0,0 wa,  0,0 hi,  0,1 si,  0,0 st
MiB Mem : 7936,7 total, 4485,5 free, 1797,6 used, 1950,0 buff/cache
MiB Swap: 4096,0 total, 4096,0 free,  0,0 used. 6139,1 avail Mem
PID to signal/kill [default pid = 3815]

```

PID	USER	PR	NI	VRT	RES	SHR	S	%CPU	%MEM	TIME+
3815	rapavli+	20	0	220988	1792	1792	R	100,0	0,0	2:09.59

Рисунок 2.4: Команда top

Получили полномочия администратора `su` - Ввели следующие команды:
`dd if=/dev/zero of=/dev/null &`
`dd if=/dev/zero of=/dev/null &`
`dd if=/dev/zero of=/dev/null &`

```
[root@rapavlichenko ~]# su -
[root@rapavlichenko ~]# dd if=/dev/zero of=/dev/null &
[1] 3914
[root@rapavlichenko ~]# dd if=/dev/zero of=/dev/null &
[2] 3915
[root@rapavlichenko ~]#
[root@rapavlichenko ~]#
[root@rapavlichenko ~]# dd if=/dev/zero of=/dev/null &
[3] 3924
```

Рисунок 2.5: Создание процессов

Ввели `ps aux | grep dd`. Это показало все строки, в которых есть буквы `dd`. Использовали PID одного из процессов `dd`, чтобы изменить приоритет. Ввели `ps fax | grep -B5 dd`

```
[root@rapavlichenko ~]# ps aux | grep dd
root      2  0.0  0.0      0  0 ?        S   14:05   0:00 [kthreadd]
rapavli+ 2427  0.0  0.3 881860 30544 ?        Ssl 14:07   0:00 /usr/libexec/evolution-addressbook-factory
rapavli+ 3085  0.1  1.3 2713560 110776 ?        Sl   14:07   0:01 /usr/lib64/firefox/firefox -contentproc -childID 2 -isForBrowser -prefsLen 34104 -prefMapSize 249431 -jsInitLen 234912 -parentBuildID 20250526151627 -greomni /usr/lib64/firefox/omni.ja -appomni /usr/lib64/firefox/browser/omni.ja -appDir /usr/lib64/firefox/browser {0933ddcf-df86-4e9c-90ef-46f8b0dc000} 2899 tab
root      3914 99.3  0.0 220988 1792 pts/0    R   14:23   2:34 dd if=/dev/zero of=/dev/null
root      3915 99.7  0.0 220988 1792 pts/0    R   14:23   2:32 dd if=/dev/zero of=/dev/null
root      3924 99.6  0.0 220988 1792 pts/0    R   14:24   1:54 dd if=/dev/zero of=/dev/null
root      3947  0.0  0.0 221820 2560 pts/0    S+  14:26   0:00 grep --color=auto dd
[root@rapavlichenko ~]# renice -n 5 <PID>
-bash: синтаксическая ошибка рядом с неожиданным маркером «newline»
[root@rapavlichenko ~]# renice -n 5 3914
3914 (process ID) old priority 0, new priority 5
[root@rapavlichenko ~]# ps fax | grep -B5 dd
  PID TTY          STAT       TIME COMMAND
    2 ?            S          0:00 [kthreadd]
```

Рисунок 2.6: Строки, где есть буквы `dd`

Нашли PID корневой оболочки, из которой были запущены процессы `dd`, и ввели `kill -9` (заменив на значение PID оболочки). Мы увидели, что наша корневая оболочка закрылась, а вместе с ней и все процессы `dd`

```
[root@rapavlichenko ~]# kill -9 3877
Убито
[root@rapavlichenko ~]#
```

Рисунок 2.7: Уничтожение процесса

Запустили команду `dd if=/dev/zero of=/dev/null` трижды как фоновое задание. Увеличили приоритет одной из этих команд, используя значение приоритета -5

```
[root@rapavlichenko ~]# dd if=/dev/zero of=/dev/null &
[1] 4057
[root@rapavlichenko ~]#

[root@rapavlichenko ~]#
[root@rapavlichenko ~]# dd if=/dev/zero of=/dev/null &
[2] 4058
[root@rapavlichenko ~]# dd if=/dev/zero of=/dev/null &
[3] 4059
[root@rapavlichenko ~]# renice -n -5 4057
4057 (process ID) old priority 0, new priority -5
[root@rapavlichenko ~]#
```

Рисунок 2.8: Задание 1

Изменили приоритет того же процесса ещё раз, но использовали на этот раз значение -15. Завершили все процессы `dd`, которые мы запустили.

```
4057 (process ID) old priority -5, new priority -15
[root@rapavlichenko ~]# jobs
[1]  Запущен      dd if=/dev/zero of=/dev/null &
[2]- Запущен      dd if=/dev/zero of=/dev/null &
[3]+ Запущен      dd if=/dev/zero of=/dev/null &
[root@rapavlichenko ~]#
[root@rapavlichenko ~]# fg 1
dd if=/dev/zero of=/dev/null
^C848971286+0 записей получено
848971285+0 записей отправлено
434673297920 байт (435 GB, 405 GiB) скопирован, 377,779 s, 1,2 GB/s

[root@rapavlichenko ~]# fg 2
dd if=/dev/zero of=/dev/null
^C848475381+0 записей получено
848475381+0 записей отправлено
434419395072 байт (434 GB, 405 GiB) скопирован, 390,844 s, 1,1 GB/s

[root@rapavlichenko ~]# fg 3
dd if=/dev/zero of=/dev/null
^C1178810878+0 записей получено
1178810878+0 записей отправлено
603551169536 байт (604 GB, 562 GiB) скопирован, 502,77 s, 1,2 GB/s
```

Рисунок 2.9: Задание 1

Запустили программу `yes` в фоновом режиме с подавлением потока вывода. Запустили программу `yes` на переднем плане с подавлением потока вывода. Приостановили выполнение программы. Заново запустили

программу `yes` с теми же параметрами, затем завершили её выполнение. Запустили программу `yes` на переднем плане без подавления потока вывода. Приостановили выполнение программы.

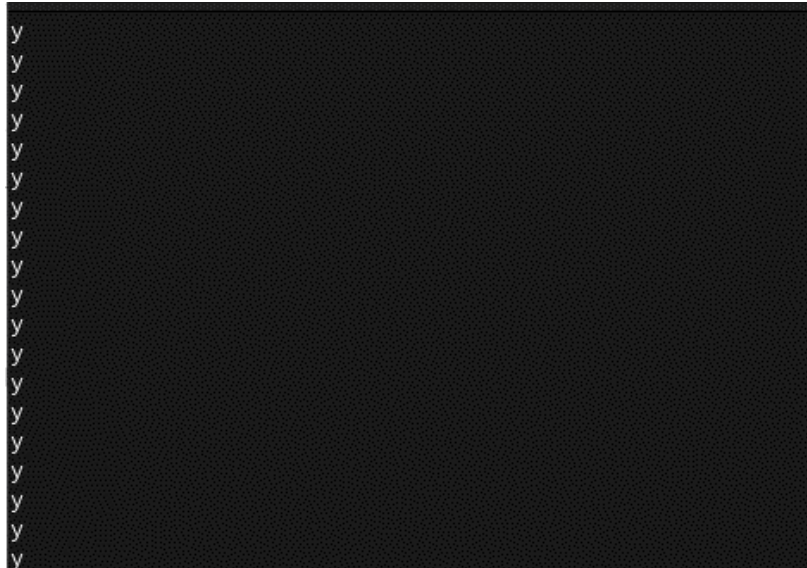


Рисунок 2.10: Задание 2

Проверили состояния заданий, воспользовавшись командой `jobs`. Перевели процесс, который у нас выполнялся в фоновом режиме, на передний план, затем остановили его.

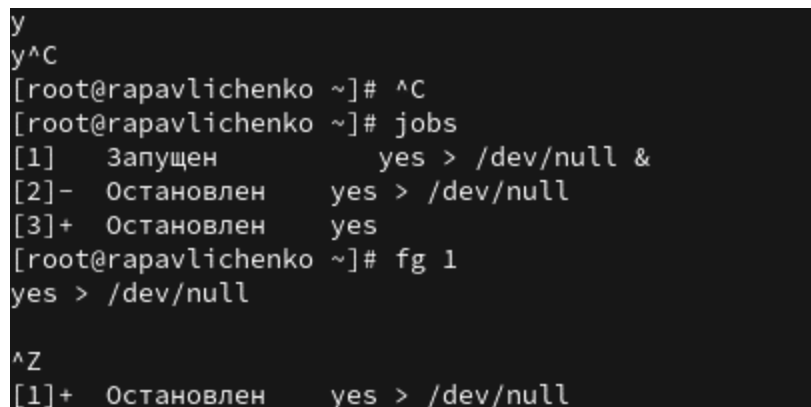


Рисунок 2.11: Задание 2

Запустили процесс в фоновом режиме таким образом, чтобы он продолжил

свою работу даже после отключения от терминала. Закрыли окно и заново запустили консоль. Убедились что процесс продолжил свою работу.

```
[root@rapavlichenko ~]# nohup yes > /dev/null &
[4] 4341
[root@rapavlichenko ~]# nohup: ввод игнорируется и поток ошибок перенаправляет
ся на стандартный вывод
[root@rapavlichenko ~]#
```

Рисунок 2.12: Задание 2

Получили информацию о запущенных в операционной системе процессах с помощью утилиты `top`. Запустили ещё три программы `yes` в фоновом режиме с подавлением потока вывода

```
 12 root      20  0      0      0      0 I  0,0  0,0  0:00.10
 13 root      20  0      0      0      0 I  0,0  0,0  0:00.00
 14 root      20  0      0      0      0 I  0,0  0,0  0:00.00
 15 root      20  0      0      0      0 I  0,0  0,0  0:00.00
 16 root      20  0      0      0      0 S  0,0  0,0  0:00.10
 18 root      20  0      0      0      0 S  0,0  0,0  0:00.00
 19 root      20  0      0      0      0 S  0,0  0,0  0:00.75
 20 root      rt  0      0      0      0 S  0,0  0,0  0:00.10
 21 root     -51  0      0      0      0 S  0,0  0,0  0:00.00
 23 root      20  0      0      0      0 S  0,0  0,0  0:00.00
 24 root      20  0      0      0      0 S  0,0  0,0  0:00.00
[root@rapavlichenko ~]# yes > /dev/null &
[5] 4413
[root@rapavlichenko ~]#
[root@rapavlichenko ~]#
[root@rapavlichenko ~]# yes > /dev/null &
[6] 4418
[root@rapavlichenko ~]# yes > /dev/null &
[7] 4424
[root@rapavlichenko ~]#
```

Рисунок 2.13: Задание 2

Запустили ещё три программы `yes` в фоновом режиме с подавлением потока вывода. Убили два процесса: для одного использовали его PID, а для другого — его идентификатор конкретного задания. Попробовали послать сигнал 1 (SIGHUP) процессу, запущенному с помощью `nohup`, и обычному процессу. Запустили ещё несколько программ `yes` в фоновом режиме с подавлением потока вывода. Завершили их работу одновременно, используя команду `killall`

```
[root@rapavlichenko ~]# killall yes
[2]  Завершено      yes > /dev/null
[4]  Завершено      nohup yes > /dev/null
[5]  Завершено      yes > /dev/null
[6]  Завершено      yes > /dev/null
[root@rapavlichenko ~]#
```

Рисунок 2.14: Задание 2

Запустили программу `yes` в фоновом режиме с подавлением потока вывода. Используя утилиту `nice`, запустили программу `yes` с теми же параметрами и с приоритетом, большим на 5. Сравнили абсолютные и относительные приоритеты у этих двух процессов. Используя утилиту `renice`, изменили приоритет у одного из потоков `yes` таким образом, чтобы у обоих потоков приоритеты были равны.

```
[root@rapavlichenko ~]# nice -n -5 yes > /dev/null &
[8] 4521
[7]  Завершено      yes > /dev/null
[root@rapavlichenko ~]# ps -l | grep yes
0 T    0    4186    3628 17  80    0 - 55237 do_sig pts/0    00:04:43 yes
0 T    0    4206    3628  0  80    0 - 55237 do_sig pts/0    00:00:00 yes
4 R    0    4521    3628 99  75   -5 - 55237 -      pts/0    00:00:22 yes
[root@rapavlichenko ~]# renice -n 5 4186
4186 (process ID) old priority 0, new priority 5
[root@rapavlichenko ~]# ps -l | grep yes
0 T    0    4186    3628 16  85    5 - 55237 do_sig pts/0    00:04:43 yes
0 T    0    4206    3628  0  80    0 - 55237 do_sig pts/0    00:00:00 yes
4 R    0    4521    3628 99  75   -5 - 55237 -      pts/0    00:00:44 yes
[root@rapavlichenko ~]#
```

Рисунок 2.15: Задание 2

3 Контрольные вопросы

1. Команда `jobs` дает обзор всех текущих заданий оболочки.
2. Остановить текущее задание, чтобы продолжить его в фоне, можно комбинацией клавиш `Ctrl+Z`, а затем командой `bg` для его запуска в фоновом режиме.
3. Комбинация клавиш `Ctrl+C` используется для отмены (прерывания) текущего задания оболочки.
4. Чтобы отменить задание без доступа к оболочке, нужно найти его PID с помощью `ps aux | grep [имя_задания]` и завершить командой `kill -9`.
5. Команда `ps tree` отображает отношения между родительскими и дочерними процессами в виде дерева.
6. Чтобы повысить приоритет процесса с PID 1234, используется команда `renice -n -p 1234` (чем меньше число, тем выше приоритет).
7. Все процессы `dd` проще всего остановить командой `pkill dd` или `killall dd`.
8. Остановить команду с именем `mycommand` можно командой `pkill mycommand`.
9. В утилите `top` для завершения процесса используется клавиша `k`, затем вводится PID процесса.
10. Чтобы запустить команду с высоким приоритетом без риска для других процессов, используйте `nice -n [команда]` (чем больше число, тем ниже приоритет, по умолчанию 10). Для уже запущенного процесса — `renice`.

4 Выводы

Получили навыки управления процессами операционной системы.

Список литературы